

ML Notebook — Program Code

Code Block 1

```
#lab-4
import pandas as pd

def explore_csv(file_path):
    print("Exploring csv files: ", file_path)
    data_csv = pd.read_csv(file_path)
    print("\nFirst five Rows:\n", data_csv.head())
    print("\nData types:\n", data_csv.dtypes)
    print("\nShape of the dataset:\n", data_csv.shape)
    print("\nColumns of dataset:\n", data_csv.columns)
    print("\nSummary Statistics:\n", data_csv.describe())
    print("\nDataframe Info:")
    data_csv.info()

def explore_excel(file_path):
    print("Exploring Excel files:\n", file_path)
    data_excel = pd.read_excel(file_path)
    print("\nFirst five Rows:\n", data_excel.head())
    print("\nData types:\n", data_excel.dtypes)
    print("\nShape of the dataset:\n", data_excel.shape)
    print("\nColumns of dataset:\n", data_excel.columns)
    print("\nSummary Statistics:\n", data_excel.describe())
    print("\nDataframe Info:")
    data_excel.info()

# Calling the functions
explore_csv('iris.csv')
explore_excel('iris.xlsx')
```

Code Block 2

```
#lab-5
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
iris = pd.read_csv('iris.csv')
sns.scatterplot(data=iris,x='petal length (cm)',y='petal width (cm)',hue='species',style='species',palette='bright')
plt.title('Petal Length vs Petal Width by Species')
plt.xlabel('Petal Length (cm)')
plt.ylabel('Petal Width (cm)')
plt.legend(title='Species')
plt.show()
sns.barplot(data=iris,x='species',y='sepal length (cm)',palette='bright')
plt.title('Average Sepal Length by Species')
plt.xlabel('Species')
plt.ylabel('Sepal Length (cm)')
plt.show()
```

Code Block 3

```
#lab-6
```

```

import pandas as pd
import numpy as np
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split

data = {
    'Age': [25, 30, np.nan, 35, 40, 22, 50, np.nan, 28, 33],
    'Salary': [50000, 60000, 70000, np.nan, 90000, 40000, np.nan, 85000, 55000, 65000],
    'Gender': ['Male', 'Female', 'Female', 'Male', 'Female', 'Male', 'Male', 'Female', 'Male', 'Female'],
    'Purchased': ['Yes', 'No', 'Yes', 'No', 'No', 'No', 'Yes', 'No', 'Yes', 'No']
}

df = pd.DataFrame(data)
print("Original DataFrame:")
print(df)

imputer_num = SimpleImputer(strategy='mean')
df[['Age', 'Salary']] = imputer_num.fit_transform(df[['Age', 'Salary']])

le_gender = LabelEncoder()
df['Gender'] = le_gender.fit_transform(df['Gender'])

le_purchased = LabelEncoder()
df['Purchased'] = le_purchased.fit_transform(df['Purchased'])

X = df.drop(columns=['Purchased'])
y = df['Purchased']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train[['Age', 'Salary']] = scaler.fit_transform(X_train[['Age', 'Salary']])
X_test[['Age', 'Salary']] = scaler.transform(X_test[['Age', 'Salary']])

print("\n Preprocessed training DataFrame:")
print(X_train.reset_index(drop=True))

print("\n The transformed target variable:")
print(y_train.reset_index(drop=True))

```

Code Block 4

```

#lab-7
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

iris = load_iris()
x = iris.data
y = iris.target

x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)

k = 3

```

```

knn_classifier = KNeighborsClassifier(n_neighbors=k)
knn_classifier.fit(x_train, y_train)

y_pred = knn_classifier.predict(x_test)

accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

print("Classification Report:")
print(classification_report(y_test, y_pred))

cm = confusion_matrix(y_test, y_pred)
print("Confusion matrix for KNN on Iris dataset:")
print(cm)

```

Code Block 5

```

#lab-8
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

data = np.array([
    [1500, 5, 200],
    [1200, 2, 180],
    [1800, 4, 220],
    [1000, 1, 160],
    [1400, 3, 190]
])

X = data[:, :2]
y = data[:, 2]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Model Coefficients (Slopes):", model.coef_)
print("Model Intercept:", model.intercept_)

print("\nModel Performance Metrics:")
print("Mean Absolute Error (MAE):", mean_absolute_error(y_test, y_pred))
print("Mean Squared Error (MSE):", mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))

new_size = float(input("Enter the size of the house: "))
new_age = float(input("Enter the age of the house: "))

predicted_price = model.predict([[new_size, new_age]])

print(predicted_price)
print(f"Predicted price for a house with size {new_size} sq.ft and age {new_age} years: {predicted_price[0]:.2f}")

```

Code Block 6

```
#lab-9
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt

iris = load_iris()
X = iris.data
Y = iris.target

X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=42)

decision_tree = DecisionTreeClassifier()
decision_tree.fit(X_train, Y_train)

Y_pred = decision_tree.predict(X_test)

accuracy = accuracy_score(Y_test, Y_pred)
print(f"Accuracy: {accuracy:.2f}")
plt.figure(figsize=(15, 10))
plot_tree(decision_tree, feature_names=iris.feature_names,
          class_names=iris.target_names, filled=True)
plt.show()
```

Code Block 7

```
#lab-10
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
import warnings
warnings.filterwarnings("ignore")

X = np.array([[30, 50000], [35, 60000], [40, 80000], [25, 30000], [45, 100000],
              [20, 20000], [50, 120000], [55, 150000], [60, 140000], [28, 40000]])

kmeans = KMeans(n_clusters=3, random_state=0)
kmeans.fit(X)

labels = kmeans.labels_
centers = kmeans.cluster_centers_

plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0], X[:, 1], c=labels, s=50, alpha=0.8)
plt.scatter(centers[:, 0], centers[:, 1], c='red', s=200, marker='x', label='Centroids')
plt.xlabel('Age')
plt.ylabel('Income')
plt.title('K-Means Clustering of Customers')
plt.legend()
plt.show()
```

Code Block 8